



آمار و احتمال مهندسی

Engineering Probability & Statistics

Lecture Notes ▪ جزوه درسی

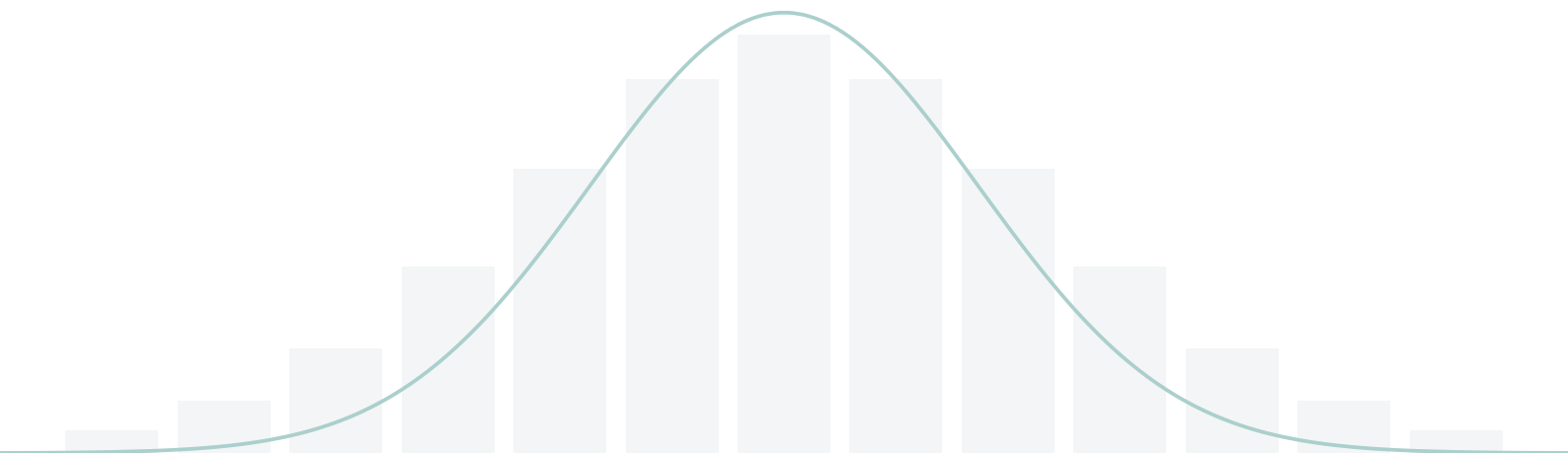
Module 16

Engineering Statistical Applications

کاربردهای آماری مهندسی

Instructor: Amin Azari

استاد: امین آذری github.com/AminAzari



Learning Objectives

This capstone module integrates ALL previous modules into real Electrical Engineering applications. Students will be able to:

1. Apply probabilistic modeling to communication systems
2. Analyze signal processing problems statistically
3. Model electronic component variations
4. Assess power system reliability
5. Handle measurement uncertainty in control systems
6. Connect probability/statistics to machine learning foundations

16.1 Communications

■ 16.1.1 BER Estimation and Confidence Intervals

Problem: Estimate the Bit Error Rate of a communication link.

Model: Each bit is a Bernoulli trial with error probability p (the BER).

- ▶ n bits transmitted, k errors observed
- ▶ $\hat{p} = k/n$ (MLE of BER)
- ▶ 95% CI: $\hat{p} \pm 1.96\sqrt{(\hat{p}(1-\hat{p}))/n}$

MATLAB: BER Simulation and CI

```
%% BER Estimation with Confidence Interval
EbN0_dB = 8; EbN0 = 10^(EbN0_dB/10);
N_bits = 1e6;

% BPSK transmission
bits = 2*randi([0,1], 1, N_bits) - 1;
noise = (1/sqrt(2*EbN0)) * randn(1, N_bits);
received = bits + noise;
decisions = sign(received);
errors = sum(decisions ~= bits);

BER_hat = errors / N_bits;
BER_theory = qfunc(sqrt(2*EbN0));

% 95% Confidence Interval
margin = 1.96 * sqrt(BER_hat*(1-BER_hat)/N_bits);
fprintf('BER estimate: %.4e\n', BER_hat);
fprintf('BER theory:   %.4e\n', BER_theory);
fprintf('95% CI: [%.4e, %.4e]\n', BER_hat-margin, BER_hat+margin);
```

MATLAB

■ 16.1.2 AWGN Noise Modeling

Additive White Gaussian Noise is the fundamental noise model:

- ▶ **Distribution:** $N(0, N_0/2)$ per dimension
- ▶ **Why Gaussian?** CLT — thermal noise is sum of many electron contributions
- ▶ **Characterization:** Fully described by power spectral density N_0 or variance $\sigma^2 = N_0 B$ (B = bandwidth)

■ 16.1.3 Rayleigh Fading Channel

Model: Channel coefficient $h = h_I + j \cdot h_Q$ where $h_I, h_Q \sim N(0, \sigma^2)$ independent.

- ▶ Envelope $|h| \sim \text{Rayleigh}(\sigma)$
- ▶ Power $|h|^2 \sim \text{Exponential}(1/(2\sigma^2))$
- ▶ **Outage probability:** $P(|h|^2 < \gamma_{th}) = 1 - \exp(-\gamma_{th}/(2\sigma^2))$

```

%% Rayleigh Fading: Outage Probability
sigma = 1;
SNR_avg_dB = 10;    % Average SNR in dB
SNR_avg = 10^(SNR_avg_dB/10);
SNR_threshold_dB = 5; % Minimum required SNR
gamma_th = 10^(SNR_threshold_dB/10);

% Analytical outage probability
P_outage = 1 - exp(-gamma_th / SNR_avg);
fprintf('Outage Probability (analytical): %.4f\n', P_outage);

% Simulation
N = 100000;
h = (randn(1,N) + 1j*randn(1,N)) / sqrt(2); % Rayleigh fading
SNR_inst = SNR_avg * abs(h).^2;
P_outage_sim = mean(SNR_inst < gamma_th);
fprintf('Outage Probability (simulated): %.4f\n', P_outage_sim);
```

MATLAB

■ 16.1.4 Rician Fading

When a line-of-sight (LOS) component exists:

- ▶ $h = h_{LOS} + h_{scatter}$
- ▶ $|h| \sim \text{Rician}(\nu, \sigma)$ where $K = \nu^2/(2\sigma^2)$ is the K-factor

K-factor: $K = 0$ (Rayleigh) $\rightarrow K = \infty$ (no fading, AWGN)

■ 16.1.5 SNR Statistics

Instantaneous SNR: $\gamma = |h|^2 \times (E_s/N_0)$

For Rayleigh: $\gamma \sim \text{Exponential}(\bar{\gamma})$ where $\bar{\gamma}$ = average SNR

Average BER over fading:

$$\bar{P}_e = \int_0^\infty P_e(\gamma) \cdot f_\gamma(\gamma) d\gamma$$

For BPSK over Rayleigh: $\bar{P}_e = (1/2)(1 - \sqrt{\bar{\gamma}/(1+\bar{\gamma})})$

16.2 Signal Processing

■ 16.2.1 Measurement Noise Characterization

Problem: Characterize noise in measured signal $x(t) = s(t) + n(t)$.

Approach:

1. Collect noise-only samples (signal absent)
2. Estimate noise statistics: $\hat{\mu}_n, \hat{\sigma}_n^2$
3. Test for Gaussianity (histogram, QQ-plot)
4. Compute SNR: $\text{SNR} = P_{\text{signal}} / \hat{\sigma}_n^2$

MATLAB

```
% Noise Characterization
fs = 1000; T = 10; % 1kHz sample rate, 10 seconds
t = 0:1/fs:T-1/fs;
N_samples = length(t);

% Simulate: known signal + noise
signal = 2*sin(2*pi*5*t); % 5 Hz sine wave
noise = 0.5*randn(1, N_samples);
measured = signal + noise;

% Estimate noise from signal-free segments (or residuals)
% Here: subtract known signal
noise_estimate = measured - signal;

fprintf('Noise Statistics:\n');
fprintf(' Mean: %.4f (should be ~0)\n', mean(noise_estimate));
fprintf(' Std: %.4f (true: 0.5)\n', std(noise_estimate));
fprintf(' SNR: %.1f dB\n', 10*log10(var(signal)/var(noise_estimate)));

% Gaussianity test
figure;
subplot(1,2,1); histogram(noise_estimate, 50, 'Normalization', 'pdf');
hold on; x = linspace(-2,2,200); plot(x, normpdf(x,0,0.5),'r','LineWidth',2);
title('Noise Histogram vs Gaussian');
subplot(1,2,2); qqplot(noise_estimate); title('QQ Plot');
```

■ 16.2.2 Correlation-Based Detection

Problem: Detect a known signal in noise.

Method: Correlate received signal with known template. Under H_0 (noise only), the correlator output has known distribution → set threshold.

MATLAB

```

%% Correlation Detector
N = 100; % Signal length
s = ones(1,N)/sqrt(N); % Known signal template (normalized)
sigma_n = 1;

% H0: noise only; H1: signal + noise
N_trials = 10000;
y_H0 = zeros(1, N_trials); y_H1 = zeros(1, N_trials);
A = 2; % Signal amplitude

for i = 1:N_trials
    n = sigma_n * randn(1, N);
    y_H0(i) = s * n'; % Correlator output under H0
    y_H1(i) = s * (A*s + n)'; % Correlator output under H1
end

% Set threshold for P_FA = 0.01
threshold = sigma_n/sqrt(N) * norminv(0.99);
P_FA = mean(y_H0 > threshold);
P_D = mean(y_H1 > threshold);
fprintf('P_FA = %.4f (target: 0.01)\n', P_FA);
fprintf('P_D = %.4f\n', P_D);

```

16.3 Electronics

■ 16.3.1 Component Tolerance Modeling

Problem: Resistors have nominal value R_0 with manufacturing tolerance.

Model: $R \sim N(R_0, \sigma_R^2)$ where σ_R depends on tolerance class.

For $\pm 5\%$ tolerance (99.7% within bounds): $3\sigma = 0.05R_0 \rightarrow \sigma = 0.0167R_0$

■ 16.3.2 Monte Carlo Circuit Analysis

Problem: A voltage divider uses R_1 and R_2 . What is the distribution of output voltage?

$$V_{\text{out}} = V_{\text{in}} \times R_2 / (R_1 + R_2)$$

With R_1, R_2 random: V_{out} is a random variable!

MATLAB

```

%% Monte Carlo: Voltage Divider with Tolerant Components
V_in = 5;           % Input voltage
R1_nom = 10e3;      R2_nom = 10e3; % 10kΩ each
tol = 0.05;         % 5% tolerance
sigma_R = tol * R1_nom / 3; % 3-sigma = 5%

N = 100000;
R1 = R1_nom + sigma_R*randn(1, N);
R2 = R2_nom + sigma_R*randn(1, N);
V_out = V_in * R2 ./ (R1 + R2);

fprintf('Nominal V_out = %.3f V\n', V_in * R2_nom/(R1_nom+R2_nom));
fprintf('Mean V_out = %.4f V\n', mean(V_out));
fprintf('Std V_out = %.4f V\n', std(V_out));
fprintf('99.7%% range: [%.3f, %.3f] V\n', ...
        mean(V_out)-3*std(V_out), mean(V_out)+3*std(V_out));

% Yield: fraction within ±2% of nominal
V_nom = 2.5;
yield = mean(abs(V_out - V_nom) < 0.02*V_nom);
fprintf('Yield (±2%%): %.2f%%\n', 100*yield);

```

■ 16.3.3 Measurement Uncertainty Propagation

If $Z = f(X, Y)$ and X, Y have small uncertainties:

$$\sigma_Z^2 \approx \left(\frac{\partial f}{\partial X} \right)^2 \sigma_X^2 + \left(\frac{\partial f}{\partial Y} \right)^2 \sigma_Y^2$$

Example: Power $P = V^2/R$. $\sigma_P^2 \approx (2V/R)^2 \sigma_V^2 + (V^2/R^2)^2 \sigma_R^2$

16.4 Power Systems

■ 16.4.1 Load Uncertainty

Problem: Peak load is uncertain. Model: $L \sim N(\mu_L, \sigma_L^2)$

Design question: What capacity C ensures $P(\text{load} > \text{capacity}) < 0.01$?

$C = \mu_L + 2.326\sigma_L$ (for 1% exceedance probability)

■ 16.4.2 System Reliability

Series system: All components must work.

$R_{\text{sys}} = R_1 \times R_2 \times \dots \times R_n$ (for independent components)

Parallel system: At least one must work.

$R_{\text{sys}} = 1 - (1-R_1)(1-R_2)\dots(1-R_n)$

MATLAB

```

%% Reliability Analysis: Series-Parallel System
% System: Two subsystems in series, each with 3 parallel redundant units
R_unit = 0.95; % Individual component reliability
n_parallel = 3;
n_series = 2;

% Each parallel subsystem
R_subsystem = 1 - (1 - R_unit)^n_parallel;
% Series combination
R_system = R_subsystem^n_series;

fprintf('Unit reliability: %.4f\n', R_unit);
fprintf('Subsystem (3 parallel): %.6f\n', R_subsystem);
fprintf('System (2 in series): %.6f\n', R_system);

% Monte Carlo verification
N = 100000;
sys_works = true(1, N);
for s = 1:n_series
    sub_works = false(1, N);
    for p = 1:n_parallel
        unit_works = rand(1, N) < R_unit;
        sub_works = sub_works | unit_works;
    end
    sys_works = sys_works & sub_works;
end
fprintf('System reliability (sim): %.6f\n', mean(sys_works));

```

16.5 Control Systems

■ 16.5.1 Sensor Noise Effects

Problem: A control system uses noisy sensor readings.

Measurement: $y(k) = x(k) + n(k)$ where $n \sim N(0, \sigma_n^2)$

Controller acts on $y(k) \rightarrow$ tracking error includes noise contribution.

Solution approaches:

- ▶ Averaging (low-pass filter): reduces noise by $\sqrt{n}$
- ▶ Kalman filter: optimal fusion of model prediction and measurement

MATLAB

```

%% Noisy Sensor in Control Loop: Effect of Averaging
true_value = 10;    % Constant being measured
sigma_n = 0.5;      % Sensor noise std dev
N_measurements = 1000;

% Raw measurements
measurements = true_value + sigma_n*randn(1, N_measurements);

% Moving average filter (window size M)
M_values = [1, 5, 10, 20, 50];
figure; hold on;
for M = M_values
    filtered = movmean(measurements, M);
    plot(filtered(100:500));
end
yline(true_value, 'k--', 'LineWidth', 2);
legend(arrayfun(@(m) sprintf('M=%d, σ=%.3f', m, sigma_n/sqrt(m)), ...
    M_values, 'UniformOutput', false));
xlabel('Sample'); ylabel('Estimate');
title('Noise Reduction by Averaging');

```

■ 16.5.2 Parameter Uncertainty

System model: $G(s) = K/(\tau s + 1)$ where K and τ are uncertain.

Monte Carlo approach: sample K and τ from their distributions, simulate system response many times, compute statistics of output.

16.6 Machine Learning / AI Connections

■ 16.6.1 Sampling and Data Collection

- ▶ Training data = sample from population
- ▶ Test data = independent sample for validation
- ▶ i.i.d. assumption underlies most ML theory

■ 16.6.2 Estimation as Foundation of ML

- ▶ Linear regression = MLE under Gaussian noise assumption
- ▶ Neural network training = MLE (or MAP) for model parameters
- ▶ Loss function minimization = maximum likelihood principle

■ 16.6.3 Probability Distributions in ML

- ▶ Generative models: learn $P(X)$ directly
- ▶ Gaussian Mixture Models: weighted sum of Gaussians
- ▶ Variational Autoencoders: latent variables with Gaussian priors

■ 16.6.4 Bias-Variance Tradeoff

$$\text{Test Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

This is exactly the MSE decomposition from estimation theory (Module 13)!

- ▶ High bias: underfitting (too simple model)
- ▶ High variance: overfitting (too complex model)
- ▶ Optimal: balance between the two

■ 16.6.5 Confidence and Uncertainty in Predictions

- ▶ Confidence intervals for model predictions
- ▶ Bayesian neural networks: posterior over weights → prediction uncertainty
- ▶ Calibration: predicted probabilities should match observed frequencies

16.7 Comprehensive MATLAB Project: Communication System Analysis

```

%% Complete Communication System Statistical Analysis
% Simulate BPSK over Rayleigh fading with AWGN
% Apply estimation, CI, and hypothesis testing

% System parameters
N_bits = 1e5;
EbN0_dB = 10;
EbN0 = 10^(EbN0_dB/10);

% Generate channel and signal
bits = randi([0,1], 1, N_bits);
symbols = 2*bits - 1; % BPSK: ±1

% Rayleigh fading channel
h = (randn(1,N_bits) + 1j*randn(1,N_bits)) / sqrt(2);
h_mag = abs(h);

% AWGN noise
noise_var = 1/(2*EbN0);
noise = sqrt(noise_var) * randn(1, N_bits);

% Received signal (with fading)
received = h_mag .* symbols + noise;

% Detection (coherent, known channel)
detected = sign(real(received ./ h_mag)); % Zero-forcing
errors = sum(detected ~= symbols);
BER_measured = errors / N_bits;

% --- Statistical Analysis ---
fprintf('=== Communication System Statistical Analysis ===\n');
fprintf('Eb/N0 = %d dB, N = %d bits\n\n', EbN0_dB, N_bits);

```

MATLAB

```

% 1. BER Estimation
fprintf('1. BER ESTIMATION\n');
fprintf('   Measured BER: %.4e (%d errors)\n', BER_measured, errors);
BER_theory_rayleigh = 0.5*(1 - sqrt(EbN0/(1+EbN0)));
fprintf('   Theory (Rayleigh): %.4e\n', BER_theory_rayleigh);

% 2. Confidence Interval
fprintf('\n2. CONFIDENCE INTERVAL (95%)\n');
margin = 1.96*sqrt(BER_measured*(1-BER_measured)/N_bits);
fprintf('   CI: [%.4e, %.4e]\n', BER_measured-margin, BER_measured+margin);
fprintf('   Theory in CI? %s\n', ...
    string(BER_theory_rayleigh >= BER_measured-margin & ...
        BER_theory_rayleigh <= BER_measured+margin));

% 3. Channel Statistics
fprintf('\n3. CHANNEL STATISTICS\n');
fprintf('   Mean |h|: %.3f (theory: %.3f)\n', mean(h_mag), sqrt(pi/4));
fprintf('   Var |h|^2: %.3f (theory: %.3f)\n', var(h_mag.^2), 1-(pi/4));

% 4. Outage Probability
SNR_threshold = 3; % dB
gamma_th = 10^(SNR_threshold/10);
P_outage_sim = mean(EbN0*h_mag.^2 < gamma_th);
P_outage_theory = 1 - exp(-gamma_th/EbN0);
fprintf('\n4. OUTAGE PROBABILITY (threshold = %d dB)\n', SNR_threshold);
fprintf('   Simulated: %.4f\n', P_outage_sim);
fprintf('   Theory:    %.4f\n', P_outage_theory);

```

16.8 Key Integration Points

Module	Application in EE
1. Probability	Event probabilities, fault diagnosis
2. Random Variables	Modeling uncertain quantities
3. Distributions	Choosing noise/fading/failure models
4. Moments	Signal power, noise power, SNR
5. Transformations	dB conversion, power from voltage
6-7. Joint/Functions	Signal+noise, system combinations
8. Conditional	Performance under different states
9. Correlation	MIMO channels, sensor arrays
10. Multiple RVs	Vector processing, beamforming
11. CLT	Why noise is Gaussian, averaging
12. Statistics	Estimating system parameters
13. Estimation	MLE for channel/noise parameters
14. Confidence	Quantifying measurement uncertainty
15. Hypothesis	Quality control, performance comparison

End of course modules. See Supplementary Materials for decision guides, MATLAB reference, and formula sheet.